

AWS Static Portfolio Website

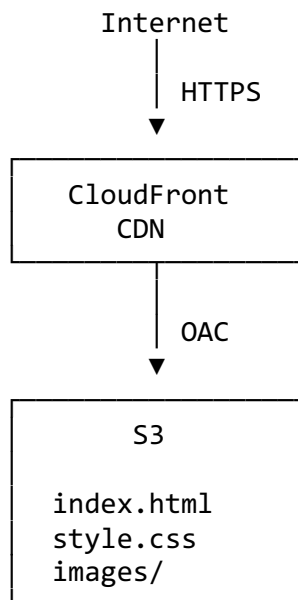
A responsive personal portfolio website designed and deployed using Amazon Web Services (AWS). The project demonstrates static website hosting, object storage, content delivery, AWS CLI deployment, and CloudFront caching.

Project Overview

The goal of this project was to gain hands-on experience with AWS by deploying a static portfolio website using Amazon S3 and Amazon CloudFront.

The website contains information about my cloud skills, AWS certification, and hands-on AWS projects that I am building as part of my cloud engineering learning journey.

Architecture



AWS Services Used

Amazon S3

Used Amazon S3 to store the static website files, including:

- HTML
- CSS
- Images

Amazon CloudFront

CloudFront is used as the content delivery layer in front of the S3 origin.

It provides:

- HTTPS delivery
- Content caching
- Global edge delivery
- Improved performance
- Cache invalidation

AWS CLI

The AWS CLI was used to synchronize local website files with the S3 bucket.

Example deployment command:

```
aws s3 sync . s3://alekhya-cloud-portfolio
```

IAM

IAM was used to manage AWS access and permissions required to work with the AWS resources.

Project Structure

```
aws-static-portfolio/  
├── index.html  
├── style.css  
├── images/  
│   └── aws-cloud-practitioner.png  
├── architecture/  
│   └── architecture.png  
├── screenshots/  
└── README.md
```

Implementation

1. Created the Portfolio Website

Built the portfolio using HTML and CSS.

The website includes:

- Introduction
- About section
- AWS skills
- AWS certification
- Hands-on projects
- Contact information

2. Created an S3 Bucket

An Amazon S3 bucket was created to store the static website files.

The website files were uploaded to the bucket and configured for static website hosting.

3. Configured Website Documents

The S3 website configuration uses:

Index document: index.html

4. Deployed Using AWS CLI

Website files can be synchronized from the local development environment to S3 using:

```
aws s3 sync . s3://alekhya-cloud-portfolio
```

This makes updating the website faster than manually uploading individual files.

5. Configured CloudFront

An Amazon CloudFront distribution was configured with the S3 bucket as the origin.

The CloudFront distribution provides HTTPS access and caches website content at edge locations.

6. Cache Invalidation

When website files are updated, CloudFront may continue serving cached versions.

A cache invalidation can be created using:

```
aws cloudfront create-invalidation \  
  --distribution-id E1BMFTMDTY8YHN\  
  --paths "/*"
```

This forces CloudFront to retrieve the updated content.

Testing

The deployment was tested by:

- Accessing the website through the CloudFront URL
- Updating HTML/CSS files
- Synchronizing updated files to S3
- Creating CloudFront invalidations
- Verifying updated content through CloudFront
- Testing website assets such as images and CSS

Challenges and Troubleshooting

During development, several common deployment issues were encountered and resolved.

Static files not loading

Investigated:

- File paths
- Object locations in S3
- HTML references
- File names and capitalization

CloudFront serving old content

CloudFront caching can cause previously cached versions of files to remain available after an update.

This was resolved by creating a CloudFront invalidation.

Security Considerations

The project was designed to use CloudFront as the public access layer for the website.

For the production architecture, the S3 bucket should remain private and CloudFront should access the S3 origin using Origin Access Control (OAC).

Administrative AWS access should be managed through IAM with the principle of least privilege.

Cost Considerations

This project was designed as a small learning project using AWS Free Tier/low-usage resources where applicable.

AWS usage should still be monitored because Free Tier eligibility and included usage vary by account and service.

The AWS Billing dashboard and Cost Explorer should be checked regularly.

Skills Demonstrated

- Amazon S3
- Amazon CloudFront
- IAM
- AWS CLI
- Static website hosting
- CDN concepts
- HTTPS
- CloudFront caching
- Cache invalidation
- Basic AWS security
- HTML/CSS
- Troubleshooting

Future Improvements

Planned improvements include:

- Custom domain using Route 53
- HTTPS with a custom domain
- Automated CI/CD deployment
- GitHub Actions
- CloudWatch monitoring
- Infrastructure as Code using Terraform
- Improved responsive design
- Performance optimization

Learning Outcomes

Through this project I gained practical experience deploying a web application to AWS rather than only learning AWS concepts theoretically.

The project helped me understand the relationship between object storage, content delivery, permissions, caching, and deployment workflows.

Project Status

Completed — initial S3 deployment and CloudFront distribution are working.

Future improvements will extend the project toward automated deployment, monitoring, and infrastructure as code.